

Campus Marketplace

Project Proposal for UCS503P

Submitted to: Dr. Raghav B. Venkataramaiyer

Thapar Institute of Engineering and Technology

Team Enigma*

August 20, 2026

1 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core user workflow and validating it with users within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback.

2 Problem Statement

Our campus (and college campuses in general) generate a constant churn of physical items, students arrive with belongings, need more as the semester goes on (especially during the first semester), and leave most of it behind or discard it when they graduate. This creates two connected but distinct problems:-

- **Inefficient buying and selling of used items:** Students frequently need affordable second-hand items (like bicycles, furniture, textbooks, appliances), but have no centralized, trustworthy platform to find them within this campus community. The current options we have are general marketplaces like Facebook Marketplace or OLX. These are not campus-specific and involve dealing with strangers off-campus, lack verification of buyers/sellers, and make logistics (meeting, payment, delivery) needlessly complicated for what should be a short walk across campus. Alternatively, students rely on social media posts or simply asking around, but these methods have limited reach and rarely produce enough options to compare. So, it makes it hard to find the best variety or the most affordable deal.

*Aditya Kaushik, Roll No: 1024030025, Email: aaditya_be24@thapar.edu,
Priyansh Singh Chouhan, Roll No: 1024030929, Email: pchouhan_be24@thapar.edu

- **No easy way to borrow rarely-used items:** Many items students own are needed only occasionally, like a drill, a gaming controller, a formal blazer for one event, project-specific equipment, etc. Buying these outright is wasteful and expensive, yet there is no structured system for students to lend and borrow such items within this campus network. As a result, students either overspend on things they will barely use or have to source it manually through word-of-mouth or social media apps.

3 Proposed Solution

3.1 Overview

We propose Campus Marketplace, a campus-exclusive platform restricted to @thapar.edu accounts that unifies two modes of resource exchange in one system: permanent buying/selling of used items and temporary borrowing/lending of items needed only occasionally. Every listing is created as either a SELL or BORROW type, each with its own pricing model (price for sale, price per day for borrowing), so the same underlying platform serves both use cases without becoming two disconnected apps.

3.2 Core Workflow

1. **Verified onboarding:** Users sign up with their Thapar email (@thapar.edu only), linked to a unique user profile before they can list, request, or borrow anything.
2. **Listing creation:** A user posts an item under a category, with a title, description, type (SELL/BORROW), price or daily rate, and photos. Once created, a listing is immutable, meaning sellers relist rather than edit their posts, keeping transaction history accurate and disputes traceable.
3. **Buying flow:** Interested buyers raise a purchase request on a listing. The owner accepts or rejects it. If accepted, it auto-generates a transaction and auto-rejects any other pending requests on that listing, since an item can only be sold once.
4. **Borrowing flow:** Interested borrowers propose a start and end time on a BORROW listing. Once accepted, a borrowing record is created with an auto-computed total amount ($price_per_day \times days$), and the system blocks any overlapping booking requests for that same listing.
5. **Hostel-based discovery:** Listings are filterable by hostels, so users can search and filter for items in their own vicinity, making handoffs quick and easy.
6. **Due-date reminders:** For active borrowings, the website sends automated reminders as the end time approaches, and a follow-up nudge if an item isn't marked returned on time, reducing the need for lenders to chase borrowers manually.
7. **Closure and trust-building:** Once a transaction is completed or a borrowing is returned, both parties can leave a rating and review of each other, building a visible trust score over time.

3.3 Operational Constraints

To keep the system consistent and abuse-resistant, a few rules are enforced at the data/business-logic level:-

- A listing can carry either a sale price or a daily rate, never both.
- Status transitions (for a borrowing or purchase request) follow a fixed state machine. An item can't jump from pending state straight to returned/completed state, skipping the intermediate states.
- Each listing can only ever produce one final transaction. The transaction amount must match the listing's price, and the recorded seller must be the item's actual owner.
- A buyer/listing pair can have at most one pending and one accepted request at a time.
- Reviews are only permitted once a deal is closed (completed transaction or returned borrowing), and only from the two actual parties involved, i.e. one review per person per deal.

4 Solution Approach

This is an engineering course project, so we focus on scalable administrative and workflow aspects rather than novel algorithm research.

- **Identity-gated access as the trust layer:** Instead of building verification systems from scratch, campus email verification is the single source of trust. It is cheap to enforce, hard to fake, and immediately scoped to the right user base.
- **State-machine-driven transactions:** Purchases and borrowings move through explicit, restricted status transitions rather than free-form updates. This makes the system's behavior predictable, auditable, and easy to extend without rewriting core logic.
- **Immutable listings for accountability:** Treating listings as append-only (relist instead of edit) keeps a clean historical record. This is useful for resolving disputes, generating trust scores, and future admin auditing, at negligible engineering cost.
- **Conflict prevention by design:** Overlapping booking checks and single transaction per listing rules prevent double-selling or double-booking at the data layer itself, rather than requiring manual moderation to catch these cases after the fact.
- **Location-aware filtering:** Hostel filtering is a simple, scalable filter on existing listing data. It improves discovery without needing any recommendation engine.
- **Reminders as scheduled jobs:** Return-date nudges can run as simple scheduled checks against end time values, keeping the borrowing flow self-managing without needing constant lender follow-up or admin intervention.
- **Thin backend, managed storage:** Node.js handles routes, business logic, and validation, while Supabase offloads authentication and file storage, reducing the surface area the team needs to build and maintain themselves, so effort stays focused on marketplace logic rather than infrastructure.

5 Engine Availability

A mature set of "engines" (tooling/services) is available to implement this as a project:-

- The frontend (React) is a static, decoupled layer that can be served independently of backend uptime even during backend maintenance, cached pages can degrade gracefully rather than fail completely.
- The backend (Node.js) is stateless by design (routes, controllers, business logic, validation). No session state lives on the server itself, so it can be horizontally scaled or restarted without losing in-flight user context.
- Supabase offloads auth and storage to a managed service with its own uptime guarantees, reducing the availability burden on the custom backend. The team isn't responsible for keeping authentication or file storage alive themselves.
- PostgreSQL is used for core data storage. Since listings are immutable and status transitions are restricted, the data layer stays consistent even under concurrent access, which matters more for reliability than raw uptime percentage at this scale.

6 Scalability

The current direction is one website per campus, each an isolated community. However, there are a few refinements we think are worth considering:-

- **Data isolation via campus scoping, not separate deployments:** Rather than literally standing up a new website per college (which multiplies infrastructure and maintenance cost), the schema can add a campus id to users and scope every query by it. One shared backend and database serve all campuses, but a Thapar user only ever sees Thapar listings. Thus it will have same operational cost as one website, and same trust boundary as separate ones.
- **Email-domain-based campus binding:** Since trust is already anchored to verified college email (@thapar.edu), each new campus just registers its email domain, and the isolation is enforced at signup. Hence, no need for separate infrastructure per campus.
- **Horizontal scaling of the backend:** Since Express handlers are stateless, adding campuses mainly adds read/write load, which can be handled by scaling backend instances behind a load balancer, rather than needing a redesign.
- **Shared category layer:** Categories can stay shared, so the team doesn't have to maintain duplicate reference data per campus.
- **Growth path:** We start with one campus to validate the workflow, then onboard additional campuses as isolated logical communities within the same system. This avoids re-architecting later and keeps a single codebase, single admin panel, and single set of integrity rules enforced everywhere.

7 Evaluation Criterion

Success can be measured across a few dimensions:-

- **Adoption:** Number of verified users per campus, active listings per week, ratio of listings created to listings actually resulting in a transaction/borrowing.
- **Trust and reliability:** Average rating/review scores, dispute rate (for e.g. items not returned, mismatched transaction amounts, etc.), percentage of borrowings returned on time versus requiring reminder escalation.
- **Efficiency of exchange:** Average time from listing creation to the request being accepted, and time from request to transaction completion.
- **Resource utilization:** Ratio of borrow transactions to sell transactions (a good split reflects the platform genuinely encouraging sharing over unnecessary buying, which supports sustainability).
- **System health:** Request latency, job success rate for reminders, and constraint-violation rate (how often the state-machine/integrity rules catch invalid transitions).

8 Higher-Order Goal

Beyond the immediate utility of “buy, sell, borrow items on campus,” the platform’s broader aim is to normalize resource-sharing as the default behavior within a trusted community, rather than an occasional favor asked of friends. By reducing the friction, risk, and search cost involved in buying second-hand or borrowing instead of purchasing new, the website aims to:

- Cut down unnecessary spending and waste generated by short-lived student needs.
- Extend the useful life of physical items by keeping them circulating within a community instead of ending up discarded when a student graduates.
- Improve trust, accountability and sustainability of the community.

9 Project Scope and deliverables

9.1 Initial Deliverable

- Core DB Schema with integrity rules enforced.
- Student login authentication and profile setup
- Create, browse and search listings (with Hostel-based filtering)
- Transaction constraints and implementation based on type of request (Buy or Borrow)

9.2 Subsequent Deliverable

- Automated return-date reminders
- Ratings and reviews post transaction/return

10 Risks and mitigations

- **Low initial adoption of the website:** Mitigated by seeding launch in our own hostel and encouraging early listings before public rollout.
- **Items not returned on time or borrower disputes:** Mitigated by automated reminders, tracking status of borrowings, and review history that flags repeat offenders.
- **Fake or duplicate accounts:** Mitigated by restricting signup to verified @thapar.edu email addresses.
- **Data inconsistency:** Mitigated by enforcing integrity rules like overlap checks and restricted status transitions directly at the database layer.
- **Users bypassing the website after first contact:** Mitigated by keeping the website flow low-friction enough to remain the easiest path, rather than something users work around.

11 Summary

Campus Marketplace is a campus-exclusive marketplace that unifies buying-selling and borrowing-lending of everyday items within a verified student community. Built on a React/Node.js/PostgreSQL stack with Supabase for auth and storage, it enforces trust and data integrity through email verification, immutable listings, and restricted status transitions rather than complex algorithms. With hostel-based filtering and automated return reminders, the platform reduces friction in peer-to-peer exchange, aiming to cut unnecessary spending and item waste while making resource-sharing a normal part of campus life, with a scoped design that can extend to multiple campuses or similar trusted communities over time.